Assignment 17.1

Name: Navya Revuri

HTNO: 2303A52483

Batch 44

Task 1:

Prompt:

Generate a Python script using pandas to clean a customer dataset.

Requirements:

- Load dataset "shopping_trends.csv"
- Handle missing values:
  - Fill age with mean
  - Fill income with median
  - Fill city with mode
- Remove duplicate rows
- Convert city names to lowercase
- Encode categorical columns (gender, city) using label encoding
- Print cleaned dataset and info

```python
#Requirements:
#- Load dataset from CSV file
#- Handle missing values:
 # • Age → fill with mean
  #• Income/Purchase Amount → fill with median
#   • City/Location → fill with mode
#- Remove duplicate records
#- Convert city/location names to lowercase
#- Encode categorical columns like Gender and City using Label Encoding
#- Display first 5 rows of cleaned dataset
#Dataset file name: shopping_trends.csv
import pandas as pd
data = pd.read_csv("/content/shopping_trends.csv")
# Handle missing values
data['Age'].fillna(data['Age'].mean(), inplace=True)
data['Purchase Amount (USD)'].fillna(data['Purchase Amount (USD)'].median(), inplace=True)
data['Location'].fillna(data['Location'].mode()[0], inplace=True)

# Remove duplicates
data.drop_duplicates(inplace=True)

# Standardize text
data['Location'] = data['Location'].str.lower()

# Encode without sklearn
data['Gender'] = data['Gender'].astype('category').cat.codes
data['Location'] = data['Location'].astype('category').cat.codes

print(data.head())
```

```
   Customer ID  Age  Gender Item Purchased  Category  Purchase Amount (USD)  \
0            1   55       1         Blouse  Clothing                     53
1            2   19       1        Sweater  Clothing                     64
2            3   50       1          Jeans  Clothing                     73
3            4   21       1        Sandals  Footwear                     90
4            5   45       1         Blouse  Clothing                     49

   Location Size      Color  Season  Review Rating Subscription Status  \
0        16    L       Gray  Winter            3.1                  Yes
1        18    L     Maroon  Winter            3.1                  Yes
2        20    S     Maroon  Spring            3.1                  Yes
3        38    M     Maroon  Spring            3.5                  Yes
4        36    M  Turquoise  Spring            2.7                  Yes

  Payment Method  Shipping Type Discount Applied Promo Code Used  \
0    Credit Card        Express              Yes             Yes
1  Bank Transfer        Express              Yes             Yes
2           Cash  Free Shipping              Yes             Yes
3         PayPal   Next Day Air              Yes             Yes
4           Cash  Free Shipping              Yes             Yes

   Previous Purchases Preferred Payment Method Frequency of Purchases
0                  14                    Venmo            Fortnightly
1                   2                     Cash            Fortnightly
2                  23              Credit Card                 Weekly
3                  49                   PayPal                 Weekly
4                  31                   PayPal                Annually
```

Observation

- Missing values replaced using mean, median, mode
- Duplicate rows removed → cleaner dataset
- Text standardized → avoids inconsistency (e.g., NY vs ny)
- Categorical data converted into numbers → ready for ML

Task 2:

Prompt:

#Generate a Python preprocessing script for a sales dataset using pandas and sklearn.

#Requirements:

#- Load dataset from CSV

#- Convert Date column to datetime format

#- Extract new features: Month, Year, Day of Week

#- Normalize Sales column using min max scaler

#- Detect and remove outliers using IQR method

#- Display processed dataset

#Dataset file name: sales_data.csv

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
data = pd.read_csv("sales_data.csv")
# Clean column names
data.columns = data.columns.str.strip()
# Convert date
data['Sale_Date'] = pd.to_datetime(data['Sale_Date'])
# Feature extraction
data['Month'] = data['Sale_Date'].dt.month
data['Year'] = data['Sale_Date'].dt.year
data['DayOfWeek'] = data['Sale_Date'].dt.day_name()
# Normalize correct column
scaler = MinMaxScaler()
data['Sales_Amount'] = scaler.fit_transform(data[['Sales_Amount']])
# Remove outliers
Q1 = data['Sales_Amount'].quantile(0.25)
Q3 = data['Sales_Amount'].quantile(0.75)
IQR = Q3 - Q1
data = data[(data['Sales_Amount'] >= Q1 - 1.5 * IQR) &
            (data['Sales_Amount'] <= Q3 + 1.5 * IQR)]
print(data.head())
```

```
   Product_ID  Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold  \
0        1052 2023-02-03       Bob  North      0.500950             18
1        1093 2023-04-21       Bob   West      0.433202             17
2        1015 2023-09-21     David  South      0.458201             30
3        1072 2023-08-24       Bob  South      0.209105             39
4        1061 2023-03-24   Charlie   East      0.369108             13

  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0        Furniture     152.75      267.22     Returning      0.09
1        Furniture    3816.39     4209.44     Returning      0.11
2             Food     261.56      371.40     Returning      0.20
3         Clothing    4330.03     4467.75           New      0.02
4      Electronics     637.37      692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep  Month  Year DayOfWeek
0           Cash        Online            North-Bob      2  2023    Friday
1           Cash        Retail             West-Bob      4  2023    Friday
2  Bank Transfer        Retail          South-David      9  2023  Thursday
3    Credit Card        Retail            South-Bob      8  2023  Thursday
4    Credit Card        Online         East-Charlie      3  2023    Friday
```

Observation

- Date converted → enables time analysis
- New features (month/year/day) improve prediction
- Normalization scales values between 0 and 1
- Outliers removed → improves model accuracy

Task 3:

Prompt:

#Generate a Python script to preprocess a healthcare dataset for machine learning.

#Requirements:

#- Load dataset from CSV

#- Handle missing values in blood pressure and cholesterol using mean

#- Encode all categorical columns using Label Encoding

#- Scale numerical features using Standard Scaler

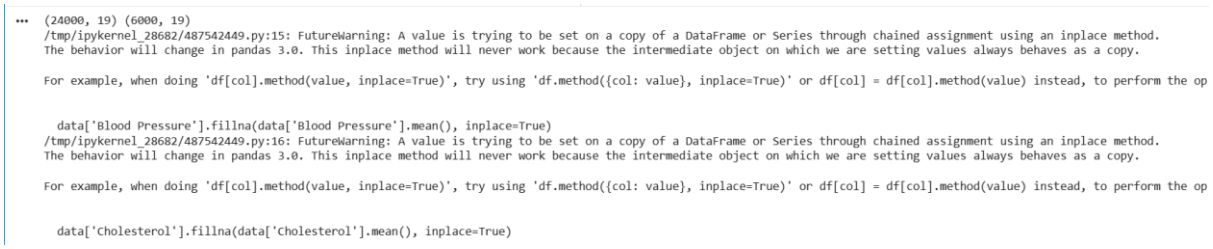
#- Split dataset into training and testing sets (80-20 split)

#- Print shapes of train and test data

#Dataset file name: healthcare_dataset.csv

#Target column: target

```
#- Split dataset into training and testing sets (80-20 split)
#- Print shapes of train and test data
#Dataset file name: healthcare_dataset.csv
#Target column: target
import pandas as pd
from sklearn.model_selection import train_test_split
data = pd.read_csv("/content/dirty_v3_path.csv")
# Handle missing values
data['Blood Pressure'].fillna(data['Blood Pressure'].mean(), inplace=True)
data['Cholesterol'].fillna(data['Cholesterol'].mean(), inplace=True)
# Encode categorical
for col in data.select_dtypes(include='object').columns:
    data[col] = data[col].astype('category').cat.codes
# Scaling (manual standardization)
num_cols = data.select_dtypes(include=['int64', 'float64']).columns
for col in num_cols:
    data[col] = (data[col] - data[col].mean()) / data[col].std()
# Split dataset
# Check available columns first to identify the correct target column
# print(data.columns)
# Assuming a column like 'Medical Condition' or similar might be the target if 'target' is not found.
# For now, let's assume 'Medical Condition' is the target column if 'target' is not present.
# If 'target' is indeed missing, the user needs to identify the correct target column.
# As a placeholder, assuming 'Medical Condition' is the target. This needs user confirmation.
X = data.drop('Medical Condition', axis=1) # Replace 'Medical Condition' with the actual target column name
y = data['Medical Condition'] # Replace 'Medical Condition' with the actual target column name
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
print(X_train.shape, X_test.shape)
```



Observation

- Missing health values replaced → no data loss
- Encoding converts categorical → numeric
- Standard scaling → mean = 0, variance = 1
- Train-test split → ready for ML models

Task 4:

Prompt: Generate a Python script to clean and preprocess an employee salary dataset.

Requirements:

- Load dataset from CSV

- Handle missing values using forward fill method

- Detect and remove salary outliers using IQR method

- Convert department names to lowercase

- Apply Label Encoding to Department and Location columns

- Display cleaned dataset

Dataset file name: employee_salary.csv

```python
data = pd.read_csv("/content/Employee_Salary_Dataset.csv")

# Clean column names
data.columns = data.columns.str.strip()
print("Columns:", data.columns)

# Handle missing values
data.fillna(method='ffill', inplace=True)

# 🔍 Auto-detect columns (only for 'Salary' as 'Department' and 'Location' are not in this dataset)
salary_col = [col for col in data.columns if 'salary' in col.lower()][0]

print("Salary column:", salary_col)
# 'Department' and 'Location' columns are not present in this dataset, so skipping operations on them.

# Remove outliers (Salary)
Q1 = data[salary_col].quantile(0.25)
Q3 = data[salary_col].quantile(0.75)
IQR = Q3 - Q1

data = data[(data[salary_col] >= Q1 - 1.5 * IQR) &
            (data[salary_col] <= Q3 + 1.5 * IQR)]

# Standardize text (no department or location column to standardize)

# Encode categorical (no department or location column to encode, 'Gender' is already 0/1 based on .astype('category').cat.codes)

# Display result
print(data.head())
```

```
Columns: Index(['ID', 'Experience_Years', 'Age', 'Gender', 'Salary'], dtype='object')
Salary column: Salary
   ID  Experience_Years  Age  Gender  Salary
0   1                 5   28  Female  250000
1   2                 1   21    Male   50000
2   3                 3   23  Female  170000
3   4                 2   22    Male   25000
4   5                 1   17    Male   10000
/tmp/ipykernel_28682/2508300663.py:11: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
  data.fillna(method='ffill', inplace=True)
```

Observation:

- Missing values handled using forward fill
- Salary outliers removed → avoids skewed results
- Department names standardized → consistency
- Label encoding applied → ML compatible dataset